P0288R9
move_only_function

Draft Proposal, 2021-07-09

Authors:
Matt Calabrese <metaprogrammingtheworld@gmail.com>
Ryan McDougall <mcdougall.ryan@gmail.com>

Audience:
LWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper proposes a conservative, move-only equivalent of `std::function`.

Table of Contents

1 Brief History

2 Overview

3 Specification

4 Polls from LEWG San Diego Review (2018)
4.1 Support func(), func() const, func() &&
4.2 Support func() && only
4.3 Remove target/target_type
4.4 Require more stuff (noexcept, const&&, ...)
4.5 Name Options

5 Polls from LEWG Kona Review (2019)
5.1 We want to spend time on this now in order to include it in C++20
5.2 Add support for func() const& and func()&
5.3 Add support for func() noexcept (x all of the above)
5.4 Include the option for CTAD
5.5 Name: callable vs any invocable
5.6 move_only_function vs invocable
5.7 Header choice
5.8 Can get std::function from <move_only_function>
5.9 Can get std::function from <invocable>
5.10 Remove the null-check in the call operator and throwing of bad_function_call
5.11 Remove the null-check in constructors that are not nullptr_t
5.12 Perfect forwarding for converting constructor instead of by-value
5.13 Forward to LWG for C++20

6 Implementation Experience

7 Acknowledgments

8 References

1. Brief History

This paper started as a proposal by David Krauss, N4543[1], from 2015 and there has been an open issue in the LEWG bugzilla requesting such a facility since 2014[2].

Since then, the paper has gone through 4 revisions and has been considered in small groups in LEWG multiple times. Gradual feedback has led to the conservative proposal seen here. The most-recent draft prior to this was a late-paper written and presented by Ryan McDougall in LEWGI in San Diego[3]. It included multiple references to implementations of move-only functions and made a strong case for the importance of a move-only form of `std::function`.

Feebdack given was encouragement for targetting C++20.

An updated version of that paper was presented on Saturday at the end of the San Diego meeting. Poll results from that presentation are presented after the overview in this document.

The revision was presented in Kona, receiving additional polls and feedback, and was to be forwarded to LWG pending updates reflecting additional poll results. Those changes have been applied to the wording in this paper. Polls from the LEWG Kona review are also provided in this document.

2. Overview

This conservative `move_only_function` is intended to be the same as `std::function`, with the exceptions of the following:

1. It is move-only.
2. It does not have the const-correctness bug of `std::function` detailed in n4348.[4]
3. It provides support for cv/ref/noexcept qualified function types.
4. It does not have the `target_type` and `target` accessors (direction requested by users and implementors).
5. Invocation has strong preconditions.

3. Specification

The following is relative to N4820.[5]

Insert the following in Header synopsis [version.syn], in section 2, below `#define __cpp_lib_memory_resource 201603L`

```
#define __cpp_lib_move_only_function 20XXXXL // also in <functional>
```

Let SECTION is a placeholder for the root of the section numbering for [functional].

Insert the following section in **Header <functional> synopsis** [functional.syn], at the end of *SECTION.16, polymorphic function wrappers*

```
template<class... S> class move_only_function; // not defined

// Set of partial specializations of move_only_function
template<class R, class... ArgTypes>
  class move_only_function<R(ArgTypes...) cv ref noexcept(noex)>; // ... see below
```

Insert the following section at the end of **Polymorphic function wrapper** [func.wrap] at the end.

```
    SECTION.16.3 move_only_function                                [func.wrap.mov]

1   The header provides partial specializations of move_only_function for
    each combination of the possible replacements of the placeholders cv, ref, and
    noex where:
```

(1.1) — *cv* is either `const` or empty.

(1.2) — *ref* is either `&`, `&&`, or empty.

(1.3) — *noex* is either `true` or `false`.

2 For each of the possible combinations of the placeholders mentioned above, there is a placeholder *inv-quals* defined as follows:

(2.1) — If *ref* is empty, let *inv-quals* be *cv*`&`

(2.2) — Otherwise, let *inv-quals* be *cv ref*.

SECTION.16.3.1 Class template move_only_function [func.wrap.mov.class]

```
namespace std {
  template<class... S> class move_only_function; // not defined

  template<class R, class... ArgTypes>
  class move_only_function<R(ArgTypes...) cv ref noexcept(noex)> {
  public:
    using result_type = R;

    // SECTION.16.3.2, construct/move/destroy
    move_only_function() noexcept;
    move_only_function(nullptr_t) noexcept;
    move_only_function(move_only_function&&) noexcept;
    template<class F> move_only_function(F&&);

    template<class T, class... Args>
      explicit move_only_function(in_place_type_t<T>, Args&&...);
    template<class T, class U, class... Args>
      explicit move_only_function(in_place_type_t<T>, initializer_list<U>, Args&&...);

    move_only_function& operator=(move_only_function&&);
    move_only_function& operator=(nullptr_t) noexcept;
    template<class F> move_only_function& operator=(F&&);

    ~move_only_function();

    // SECTION.16.3.3, move_only_function invocation
    explicit operator bool() const noexcept;

    R operator()(ArgTypes...) cv ref noexcept(noex);

    // SECTION.16.3.4, move_only_function utility
    void swap(move_only_function&) noexcept;

    friend void swap(move_only_function&, move_only_function&) noexcept;

    friend bool operator==(const move_only_function&, nullptr_t) noexcept;

  private:
    template<class VT>
      static constexpr bool is-callable-from = see below; // exposition-only
  };
}
```

1 The `move_only_function` class template provides polymorphic wrappers that generalize the notion of a callable object [func.def]. These wrappers can store, move, and call arbitrary callable objects, given a call signature, allowing functions to be first-class objects.

2 Implementations are encouraged to avoid the use of dynamically allocated memory for a small contained value.

[Note: Such small-object optimization can only be applied to types `T` for which `is_nothrow_move_constructible_v<T>` is `true`. -- end note]

SECTION.16.3.3 Constructors and destructor [func.wrap.mov.con]

```
template<class VT>
    static constexpr bool is-callable-from = see below; // exposition-only
```

1 If *noex* is `true`, `is-callable-from<VT>` is equal to
`is_nothrow_invocable_r_v<R, VT` *cv ref*`, Args...> &&`
`is_nothrow_invocable_r_v<R, VT` *inv-quals*`, Args...>.`
Otherwise, `is-callable-from<VT>` is equal to
`is_invocable_r_v<R, VT` *cv ref*`, Args...> &&`
`is_invocable_r_v<R, VT` *inv-quals*`, Args...>.`

```
move_only_function() noexcept;
move_only_function(nullptr_t) noexcept;
```

2 *Postconditions:* `*this` has no target object.

```
move_only_function(move_only_function&& f) noexcept;
```

3 *Postconditions:* The target object of `*this` is the target object `f` had before construction, and `f` is in a valid state with an unspecified value.

```
template<class F> move_only_function(F&& f);
```

4 Let `VT` be `decay_t<F>`.

5 *Constraints:*

(5.1) — `remove_cvref_t<F>` is not the same type as `move_only_function`, and

(5.2) — `remove_cvref_t<F>` is not a specialization of `in_place_type_t`, and

(5.3) — `is-callable-from<VT>` is `true`.

6 *Mandates:* `is_constructible_v<VT, F>` is `true`

7 *Preconditions:* `VT` meets the *Cpp17Destructible* requirements, and if `is_move_constructible_v<VT>` is `true`, `VT` meets the *Cpp17MoveConstructible* requirements.

8 *Postconditions:* `*this` has a target object unless any of the following hold:

(8.1) — `f` is a null function pointer value, or

(8.2) — `f` is a null member pointer value, or

(8.3) — `remove_cvref_t<F>` is a specialization of the `move_only_function` class template,
and `f` has no target object.

9 Otherwise, `*this` targets an object of type `VT` direct-non-list-initialized with `std::forward<F>(f)`.

10 *Throws:* Any exception thrown by the initialization of the target object. May throw `bad_alloc` unless `VT` is a function pointer or a specialization of `reference_wrapper`.

```
template<class T, class... Args>
  explicit move_only_function(in_place_type_t<T>, Args&&... args);
```

11 Let `VT` be `decay_t<T>`.

12 *Constraints:*

(12.1) — `is_constructible_v<VT, Args...>` is `true`, and

(12.2) — `is-callable-from<VT>` is `true`.

13 *Mandates:* `VT` is the same type as `T`.

14 *Preconditions:* `VT` meets the *Cpp17Destructible* requirements, and if `is_move_constructible_v<VT>` is `true`, `VT` meets the *Cpp17MoveConstructible* requirements.

15 *Postconditions:* `*this` targets an object of type `VT` direct-non-list-initialized with `std::forward<Args>(args)...`.

16 *Throws:* Any exception thrown by the initialization of the target object. May throw `bad_alloc` unless `VT` is a function pointer or a specialization of `reference_wrapper`.

```
template<class T, class U, class... Args>
  explicit move_only_function(in_place_type_t<T>, initializer_list<U> ilist, Args&&... args);
```

17 Let `VT` be `decay_t<T>`.

18 *Constraints:*

(18.1) — `is_constructible_v<VT, initializer_list<U>&, ArgTypes...>` is `true`, and

(18.2) — `is-callable-from<VT>` is `true`.

19 *Mandates:* `VT` is the same type as `T`.

20 *Preconditions:* `VT` meets the *Cpp17Destructible* requirements, and if `is_move_constructible_v<VT>` is `true`, `VT` meets the *Cpp17MoveConstructible* requirements.

21 *Postconditions:* `*this` targets an object of type `VT` direct-non-list-initialized with `ilist, std::forward<ArgTypes>(args)...`.

22 *Throws:* Any exception thrown by the initialization of the target object. May throw `bad_alloc` unless `VT` is a function pointer or a specialization of `reference_wrapper`.

```
move_only_function& operator=(move_only_function&& f);
```

23 *Effects:* Equivalent to: `move_only_function(std::move(f)).swap(*this);`

24 *Returns:* `*this`.

```
move_only_function& operator=(nullptr_t) noexcept;
```

25 *Effects:* Destroys the target object of `*this`, if any.

26 *Returns:* `*this`.

```
template<class F> move_only_function& operator=(F&& f);
```

27 *Effects:* Equivalent to: `move_only_function(std::forward<F>(f)).swap(*this);`

28 *Returns:* `*this`.

```
~move_only_function();
```

29 *Effects:* Destroys the target object of `*this`, if any.

mov.inv]

```
explicit operator bool() const noexcept;
```

1 *Returns:* `true` if `*this` has a target object, otherwise `false`.

```
R operator()(ArgTypes... args) cv ref noexcept(noex);
```

2 *Preconditions:* `*this` has a target object.

3 *Effects:* Equivalent to:

```
return INVOKE<R>(static_cast<F inv-quals>(f), std::forward<ArgTypes>(args)...);
```

where `f` is the target object of `*this` and `f` is lvalue of type `F`.

SECTION.16.3.5 Utility [func.wrap.mov.util]

```
void swap(move_only_function& other) noexcept;
```

1 *Effects:* Exchanges the targets of `*this` and `other`.

```
friend void swap(move_only_function& f1, move_only_function& f2) noexcept;
```

2 *Effects*: Equivalent to: `f1.swap(f2)`.

```
friend bool operator==(const move_only_function& f, nullptr_t) noexcept;
```

3 *Returns:* `true` if `f` has no target object.

4. Polls from LEWG San Diego Review (2018)

4.1. Support func(), func() const, func() &&

SF	F	N	A	SA
6	6	2	1	0

4.2. Support func() && only

SF	F	N	A	SA
2	2	7	1	1

4.3. Remove target/target_type

SF	F	N	A	SA
12	5	0	0	0

4.4. Require more stuff (noexcept, const&&, ...)

SF	F	N	A	SA
0	1	8	6	0

Note that the final poll (require more stuff) was not due to members being against the design, but because we could easily add those facilities in a later standard without any breakage.

4.5. Name Options

There was one final poll, which brought us to the name `any_invocable`.

```
3  unique_function
3  move_function
2  move_only_function
7  movable_function
8  mfunction
10 any_invocable
8  mofun
8  mofunc
0  callback
4  mvfunction
2  func
0  callable
2  any_function
```

5. Polls from LEWG Kona Review (2019)

5.1. We want to spend time on this now in order to include it in C++20

```
SF F N A SA
8  8 2 0 0
```

5.2. Add support for func() const& and func()&

```
SF F N A SA
0  8 7 0 0
```

5.3. Add support for func() noexcept (x all of the above)

```
SF F  N A SA
2  12 2 0 0
```

5.4. Include the option for CTAD

```
SF F N A SA
0  1 5 9 0
```

5.5. Name: callable vs any invocable

```
SC C N AI SAI
3  2 3 5  6
```

5.6. any_invocable vs invocable

```
SAI AI N I SI
3   7  2 5 1
```

5.7. Header choice

```
7 <functional>
11 <any_invocable>
11 <invocable>
3 <()>
```

5.8. Can get std::function from <any_invocable>

```
SF F N A SA
0  1 4 4 7
```

5.9. Can get std::function from <invocable>

```
SF F N A SA
1  3 6 3 2
```

Decide on <any_invocable>. Unanimous for <functional> to pull it in, even if in its own header.

5.10. Remove the null-check in the call operator and throwing of bad_function_call

```
SF F N A SA
8  2 1 0 0
```

5.11. Remove the null-check in constructors that are not nullptr_t

```
std::any_callable ac = my_ptr_object;
if(ac)  { /* true even if my_ptr is nullptr */ }
```

```
SF F N A SA
0  2 2 4 3
```

5.12. Perfect forwarding for converting constructor instead of by-value

Unanimous

5.13. Forward to LWG for C++20

```
SF F N A SA
8  5 0 0 0
```

6. Implementation Experience

There are many implementations of a move-only `std::function` with a design that is similar to this. What is presented is a conservative subset of those implementations. The changes suggested in LEWG, though minimal, have not been used in a large codebase.

Previous revisions of this paper have included publicly accessible move-only function implementations, notably including implementations in HPX, Folly, and LLVM.

§ 7. Acknowledgments

Thanks to Tomasz Kamiński, Tim Song, and Nevin Liber for suggestions on wording simplifications.

§ 8. References

[1]: David Krauss: N4543 "A polymorphic wrapper for all Callable objects" http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4543.pdf

[2]: Geoffrey Romer: "Bug 34 - Need type-erased wrappers for move-only callable objects" https://issues.isocpp.org/show_bug.cgi?id=34

[3]: Ryan McDougall: P0288R2 "The Need for std::unique_function" https://wg21.link/p0288r2

[4]: Geoffrey Romer: N4348 "Making std::function safe for concurrency" www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4348.html

[5]: Richard Smith: N4820 "Working Draft, Standard for Programming Language C++" http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/n4820.pdf